

配布資料の内容

5.1 繰り返しの処理 (続き)	5-1
5.2 演習問題	5-6
5.3 今回の実習内容	5-7
5.4 付録: タートルグラフィックスの機能一覧	5-8

5.1 繰り返しの処理 (続き)

今回は、引き続き、繰り返しの処理に関するプログラミングの考え方について勉強します。

フラグ 次のようなプログラムを作ることを考えます。

キーボードから正の整数を入力すると、それが素数であるかどうかを判定して表示するプログラム (`prime.c`)

素数とは、1 と自分自身以外の正の整数では割り切れないような 2 以上の整数のことです。このプログラムをコンパイルして実行したら、次の例のように動かなければなりません。

素数でない数が入力された場合

 正の整数 => 38
38は素数ではありません

素数が入力された場合

 正の整数 => 37
37は素数です

正の整数が素数であるかどうかを判定するためには、次のような仕事を行えばよいはずですが。

1. 正の整数をキーボードから入力してもらい、それを変数 (たとえば `n`) に格納しておく。
2. 2 から `n-1` までの整数で割ったときの余りをそれぞれ計算する。
3. `n` が 1 以下であるか、あるいは (手順 2 の) いずれかの数で割ったときの余りが 0 ならば「素数でない」と出力し、どの数で割ったときの余りも 0 でなければ「素数です」と出力する。

この内、手順の 2 は繰り返しの処理 (`while` 文) になりますし、手順の 3 は場合分けの処理 (`if` 文) になります。では、「(手順 2 の) いずれかの数で割ったときの余りが 0」という条件をどうやって調べたらよいのでしょうか? ある変数の値がある定数に等しいとか、ある定数より大きいとかいった類の条件であれば、それをそのまま `if` 文の条件式の部分に書けばよいわけですが、「いずれかの数で割ったときの余りが 0」という条件をそのまま `if` 文の条件式の部分に書くことはできません。「割ったときの余りが 0」ということは、あくまで 2 から `n-1` までのそれぞれの整数に対して (手順 2 の繰り返しの処理の中で) 分かるだけです。このような場合、次のように考えてプログラムを作るとうまく行きます。

- 繰り返しの処理を始める前に、ある変数 (たとえば `found`) に 0 を格納しておく。

- 手順2の繰り返しの処理では、それぞれの数で割ったときの余りが0だった場合に、その印として変数 `found` に1を代入する。
- 手順3の段階で、変数 `found` に格納されている値が1の場合は「いずれかの数で割ったときの余りが0」と解釈し、0の場合は「どの数で割ったときの余りも0でない」と解釈する。

どの数で割っても割りきれない(余りが0でない)場合は、変数 `found` には繰り返しの処理を始める前に代入した0が格納されたままのはずですし、いずれかの数で割りきれた(余りが0だった)のなら、そこで変数 `found` の値は1に変わってしまっているはず。

メモ

変数 `found` は「それまでに割りきれた数があるかどうか」という情報を保持していると考えることができます。変数 `found` が1ならそのような数があったわけですし、0ならまだそのような数は見つかっていないということです。この `found` のように、何かの仕事を繰り返して(続けて)ゆく中で、ある特定の状況がすでに起ったかどうか(ある特定の条件がすでに成り立ったかどうか)を示すために使われる変数を「フラグ(旗)」と呼びます。その特定の状況が起ったときに変数の内容を書き換えることを「フラグを立てる」といいます。この例では、まず最初に `found` に0を代入して(フラグを降ろして)おき、「ある数で `n` が割りきれた」という状況でフラグを立てています。このフラグは、`while` 文による繰り返しの処理がすべて完了した時に調べられ、そこで場合分けの処理を行うわけです。

次は、以上のような考えに沿って書かれたプログラムです。

prime.c

```
#include <stdio.h>

int main()
{
    int n, k, found;

    printf("正の整数 => ");
    scanf("%d", &n);
    found = 0;
    k = 2;
    while (k < n) {
        if (n % k == 0)
            found = 1;
        k = k+1;
    }
    if (n <= 1 || found == 1)
        printf ("%dは素数ではありません\n", n);
}
```

素数でない数が入力された場合



正の整数 => 38
38は素数ではありません

素数が入力された場合



正の整数 => 37
37は素数です

```
    else
        tPrintf ("%dは素数です\n", n);
}
```

繰り返しの処理の途中終了 prime.c のプログラムの行っている仕事をよく観察すると、一部無駄な仕事を行っていることが分かります。2 から順に $n-1$ までの数で n を割ってみていますが、一度ある数で割りきれた (フラグを立てた) のなら、それ以上の数についてさらに調べていく必要は無いはずです。このような考えに基づいて繰り返しの条件を書き換え、以下のようなプログラムにすることもできます。

```
found = 0;
k = 2;
while (found == 0 && k < n) {
    if (n % k == 0)
        found = 1;
    k = k+1;
}
```

つまり、フラグが立っていない (まだ割り切れる数が見つかっていない) 場合に限り繰り返しの続けるわけです。

メモ

break 文による繰り返しからの脱出 先の例では while 文の条件式を工夫することで、不要な繰り返しの処理を取り除きましたが、C ではもっと直接的な方法で繰り返しの処理の途中終了を行うことができます。次のプログラムを見てください。

```
found = 0;
k = 2;
while (k < n) {
    if (n % k == 0) {
        found = 1;
        break;
    }
    k = k+1;
}
```

このプログラムでは、while 文の条件式は prime.c と変わっていません。変わっているのは n が k で割りきれたときに実行される if 文の中です。変数 found のフラグを立てているのは同じですが、それに引き続いて break; という文を実行しています。

この `break` は `if` や `while` と同じように C の予約語の 1 つで、繰り返しの処理などを途中で終了するときに用います。 `while` 文の中で `break;` が実行されると、そこで `while` 文全体の実行が終了します。そこで繰り返しの処理が終るわけです。二重三重の `while` 文の中で `break;` が実行された場合は、一番内側の (つまりその `break` 文の 1 つ外側の) `while` 文の実行が終了します。

`prime.c` を `break` 文を使ってこのように書き換えると、フラグとして使っていた `found` という変数も必要が無くなっていることに気づきます。変数 `k` を 2 から順に $n-1$ まで動かして、その数で n が割りきれるかどうかを調べているわけですから、最後まで割りきれなかった場合には `k` の値が n と等しくなって繰り返しの処理が終わっているはずですし、もし途中の数で割りきれってしまったのなら、`k` は n に達する前に (つまり $k < n$ が成り立っている状態で) 繰り返しの処理が (`break` 文によって) 終了しているはずですよ。ですから `while` 文の後で $k < n$ が成り立っているかどうかを調べることで「いずれかの数で割ったときの余りが 0」かどうかを判定することができます。このような考えに基づいて `prime.c` を書き直すと次のようなプログラムになります。

smartprime.c

```
#include <turtle.h>

int main()
{
    int n, k;

    tPrintf("正の整数 => ");
    tScanf("%d", &n);
    k = 2;
    while (k < n) {
        if (n % k == 0)
            break;
        k = k+1;
    }
    if (n <= 1 || k < n)
        tPrintf ("%dは素数ではありません\n", n);
    else
        tPrintf ("%dは素数です\n", n);
}
```

素数でない数が入力された場合

 正の整数 => 38
38は素数ではありません

素数が入力された場合

 正の整数 => 37
37は素数です

この `smartprime.c` も `prime.c` と同じように正しく素数かどうかを判定して出力してくれます。

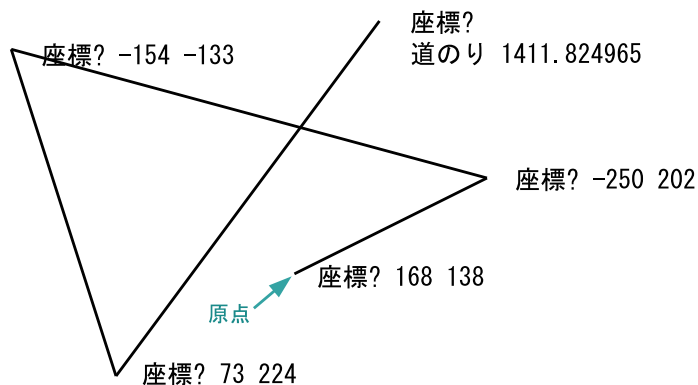
メモ

break 文の応用 `break` 文はいろいろと応用が効きます。つぎの `route.c` は、`tScanf` を使ってキーボードから x 座標と y 座標の値を読み込み、カメを指定された点に次々と移動させていくプロ

グラムです。C 言語では、繰り返しの条件として () の中に定数の 1 を書いておくと、while 文は永遠に繰り返しを続けるようになります。tScanf という関数は、戻り値としてキーボードから読み取ることのできたデータの個数を整数値として返しますので、この数が 2 でない場合に、break 文を使って繰り返しを終了し、これまでの移動の道のり (合計) を表示するようにしています。

```
route.c
1 #include <turtle.h>
2
3 int main()
4 {
5     double x, y, sum;
6
7     sum = 0;
8     while (1) {
9         tPrintf("座標? ");
10        if (tScanf("%lf%lf", &x, &y) != 2)
11            break;
12        sum = sum + tMoveTo(x, y);
13    }
14    tPrintf("道のり %f\n", sum);
15 }
```

下図は、このプログラムを起動後、キーボードから「168 138」、「-250 202」、「-154 -133」、「73 224」の順に入力し、最後に、何も入力せずに Enter キーを押した場合の実行例です。



tScanf の関数呼び出しによって表示された箱に何も入力せずに Enter キーを押すと、戻り値は 0 となりますので、プログラム 11 行目の break 文が実行されて、while 文による繰り返しの処理がそこで終了します。

メモ

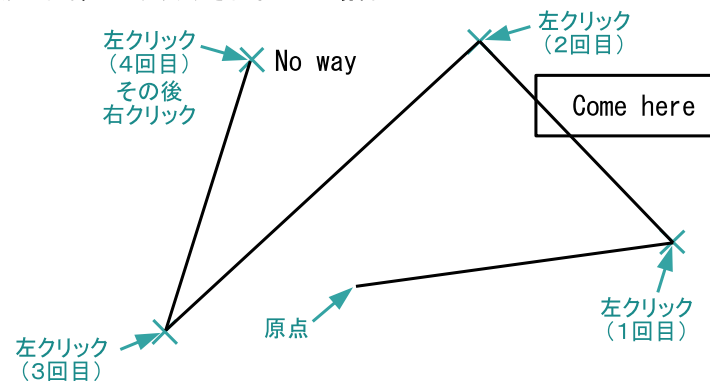
5.2 演習問題 — 注意：AI にプログラムを作成してもらうのはだめです

1. つぎのようなプログラム `comehere.c` を自分の力で作成し、コンパイル、実行して、正しく動作することを確認しなさい。

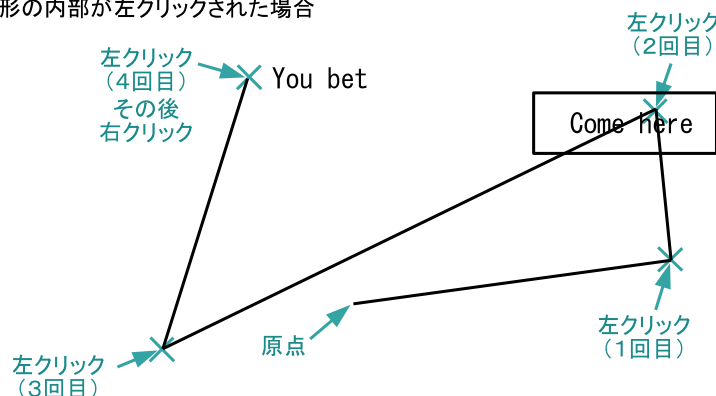
- (1) まず、(100, 80)、(100, 120)、(200, 120)、(200, 80) の4つの点を頂点とする長方形を描き、その中に「Come here」という文字列を書く。
- (2) ペンを上げて、原点に戻る。
- (3) 左クリックされる度に、クリックされた点に向けて線分を描く、という仕事を繰り返す。
- (4) 左ボタン以外でクリックされたら (3) の繰り返しをやめて、それまでに (1) の長方形の内部 (辺上は含まない) が左クリックされたことがあったのなら「You bet」という文字列を、なかったのなら「No way」という文字列を (その場で) 表示する。

ただし、(1) の「Come here」という文字列は、カメを (100, 100) に位置させて書いてください。

長方形の内部が左クリックされなかった場合



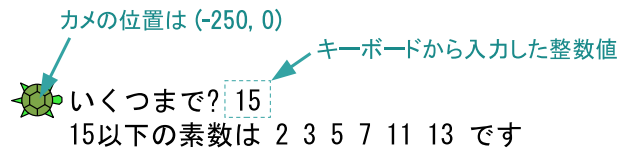
長方形の内部が左クリックされた場合



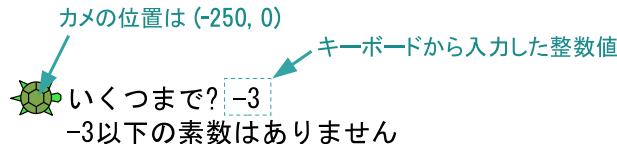
ヒント： (1) の長方形の内部がクリックされたかどうかを、フラグを使って記憶するようにしましょう。

2. キーボードから整数 n を入力すると、 n 以下の素数を次の実行例のように出力するプログラム `primes.c` を自分の力で作成し、コンパイル、実行して、正しく動作することを確認しなさい。ただし、たくさんの素数を表示できるように、まず、カメを $(-250, 0)$ の位置に (ペンを上げた状態

で)移動してから始めるようにしてください。このプログラムが表示する「いくつまで?」の右や、「2」、「3」、「5」、... などの素数の両側には半角のスペースが1つあります。



また、キーボードから入力されたのが1以下の整数であった場合、つぎのように「... 以下の整数はありません」と表示するようにしなさい。



ヒント: 2から n までの各自然数 m に対して、順に m が素数かどうかを判定し、素数であれば m の値を表示していきましょう。例題の `smartprime.c` のやり方で、ある数が素数であるかどうかを判定することができます。全体は2重の繰り返しの処理になるはずですが、 m を1ずつ増やしていく繰り返しの中に、2から $m-1$ までの各自然数 k で m が割りきれるかどうかを調べていく繰り返しがあることになります。

5.3 今回の実習内容

1. プリントをもう一度読み返しましょう。`prime.c`、`smartprime.c`、`route.c` の3つの例題プログラムについては、それぞれテキストエディタを使ってソースファイルを作成し、コンパイル、実行して、正しく動作することを確認してください。プログラムが完成したら「課題の提出と確認」の Web ページから提出してください。
2. 演習問題に取り組みましょう。それぞれのプログラムが完成したら、「課題の提出と確認」の Web ページからの提出を忘れないでください。
3. クイズに答えてください。前回までと同様に「課題の提出と確認」の Web ページで「第5回クイズ」を選択し、「送信」のボタンをクリックしてクイズに答えてください。
4. 付録の「タートルグラフィックスの機能一覧」にある関数を使って、いろいろな絵を描く(動作をする)プログラムを作成してみましょう。次回以降の課題として、自由に魅力的なプログラムを作成して頂く予定ですので、タートルグラフィックスのいろいろな機能を試しておいてください。

5.4 付録: タートルグラフィックスの機能一覧

関数の呼び出し方	働き	戻り値
tForward(d)	実数値 d だけ前へ進む	進んだ距離 d (実数値)
tBackward(d)	実数値 d だけ後ろへ戻る	戻った距離 d (実数値)
tTurn(t)	反時計回りに t° だけ向きを変える (t は実数値)	x 軸の正の向きを 0° としたときの カメの新しい向き (実数値)
tMoveTo(x, y)	実数値の座標 (x, y) へ移動する	移動した距離 (実数値)
tPenUp()	ペンを上げる	変更前にペンを上げていれば 0、下 げていれば 1 (整数値)
tPenDown()	ペンを下げる	
tGetClick()	マウスボタンがクリックされるのを待つ	クリックされたボタンの番号 (整数 値 — 左=1、中=2、右=3)
tCheckClick(t)	マウスボタンがクリックされるのを最大 t 秒待つ (t は実数値)	クリックされたボタンの番号 (整数 値 — クリックされなかった=0、左 =1、中=2、右=3)
tClickButton()	特に何もしない	
tClickCount()	特に何もしない	連続してマウスクリックされた回数 (整数値)
tClickX()	特に何もしない	マウスクリックされた位置の x 座 標 (整数値)
tClickY()	特に何もしない	マウスクリックされた位置の y 座 標 (整数値)
tPrintf(f, ...)	出力書式文字列 f と残りの引数で指定 された文字列を、カメの右側に表示す る	なし
tScanf(f, ...)	キーボードから入力された数値等を入力 書式文字列 f で解釈し、残りの引数 で指定された変数等に格納する (変数 の前に & が必要)	読み込めた数値等の個数 (整数値)
tSetColor(r, g, b)	光の3原色 (赤、緑、青) の各成分の強 さを 0.0 ~ 1.0 の範囲の実数値 r、g、 b でそれぞれ指定して、ペンや塗り潰 し、文字列の色を変更する	変更前の色を表す整数値 (赤、緑、 青の強さを、それぞれ 0 ~ 255 の範 囲の整数 R、G、B で表したときの $65536R + 256G + B$)
tSetBackground (r, g, b)	tSetColor と同様の方法で、ウィンド ウの背景色を変更する	
tFill()	tMark、または tFill や tFillAll を 最後に呼び出した時点から現在までの カメの移動 (ペンを上げていてもよい) の軌跡の内部を塗り潰す ¹	なし
tFillAll()		
tMark()	tFill や tFillAll で使われる軌跡の 始まりをカメの現在位置に設定する	なし
tCircle(r)	反時計回りに半径 r の円周上を一周す る — 半径 r が負の場合は時計回りと なる (r は実数値)	移動した距離 (実数値)

¹軌跡の内部が複数の領域に分割される場合、tFill は、外側のものから順に、塗り潰す領域と塗り潰さない領域を交互に設定するが、tFillAll はすべての領域を塗り潰す。

関数の呼び出し方	働き	戻り値
tArc(r, t)	反時計回りに半径 r の円周上を t° だけ移動する — 半径 r が負の場合は時計回りとなる (r, t は実数値)	移動した距離 (実数値)
tHeadTo(x, y)	実数値の座標 (x, y) の方向へ向きを変える	x 軸の正の向きを 0° としたときのカメラの新しい向き (実数値)
tTurnTo(t)	x 軸の正の向き基準として、反時計回りに t° の方向へ向きを変える (t は実数値)	
tSetSpeed(s)	カメラの移動速度を、毎秒 s ピクセルに設定する (初期値は毎秒 200 ピクセル) — s が 0.0 の場合は最高速度に設定する (s は実数値)	変更前の移動速度 (実数値)
tSetWidth(w)	カメラが描く軌跡の線幅を w ピクセルに設定する (w は整数値)	変更前の線幅 (実数値)
tSetFontSize(s)	文字列の描画に使われる文字の大きさ (高さ) を s (整数値) ピクセルに設定する (s の初期値は 16)	変更前の文字の大きさ (整数値)
tHide()	カメラを表示しないようにする (表示されていなくてもカメラは仕事をすることができる)	変更する前に表示されていなければ 0、されていれば 1 (整数値)
tShow()	カメラを表示する	
tSetSize(s)	カメラの大きさを s ピクセルに設定する (初期値は 20 ピクセル)	変更前の大きさ (実数値)
tSetShellColor (r, g, b)	光の 3 原色 (赤、緑、青) の各成分の強さを $0.0 \sim 1.0$ の範囲の実数値 r, g, b でそれぞれ指定して、カメラの甲羅の色を変更する	変更前の色を表す整数値 (赤、緑、青の強さを、それぞれ $0 \sim 255$ の範囲の整数 R, G, B で表したときの $65536R + 256G + B$)
tX()	特に何もしない	カメラの現在位置の x 座標 (実数値)
tY()	特に何もしない	カメラの現在位置の y 座標 (実数値)
tOrientation()	特に何もしない	x 軸の正の向きを 0° としたときのカメラの現在の向き (実数値)
tCancel(n)	直前の n 個の描画処理 (tMark を含む) を取り消す	なし
tClear()	これまでの描画処理 (tMark を含む) をすべて取り消す	なし
tErase(n, m)	n 個前から m 個前までの描画処理で描かれた部分を消去する (tMark やカメラの移動の軌跡は取り消されない)	なし
tPause(t)	プログラムの実行を t 秒間停止する (t は実数値)	なし
tMaximizeWindow()	ウィンドウを最大化する	なし
tNormalizeWindow()	ウィンドウを通常の高さに戻す	なし
tResizeWindow(w, h)	ウィンドウの描画領域の幅を w ピクセルに、高さを h ピクセルにする (w, h は整数値)	なし

付録: 前回の演習問題のプログラム例

sure.c

```
#include <turtle.h>

int main()
{
    tPrintf("He said, \"I'm 100%% sure.\\n");
    tPrintf("But I doubted it.");
}
```

vfollow.c

```
#include <turtle.h>

int main()
{
    int x, y;

    while (tGetClick() == 1) {
        x = tClickX();
        y = tClickY();
        tPrintf("(%d,%d)\n距離 = %.1f", x, y, tMoveTo(x, y));
    }
    tPenUp();
    tMoveTo(0, 0);
}
```

asterisks.c

```
#include <turtle.h>

int main()
{
    int i, k, n;

    tPrintf("行数? ");
    tScanf("%d", &n);
    k = 0;
    while (k < n) {
        k = k + 1;
        i = 0;
        while (i < k) {
            tPrintf("*");
            i = i + 1;
        }
        tPrintf("\n");
    }
}
```